pfi

Python for Fixed Income

COURSE 7 · NLP, RAG, AND LLMS

Course 7 Capstone

WEEK 3 · DAY 5

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The scenario: a table a person can check

A colleague on the desk: "Build me a tool that takes an indenture in and gives back the covenant terms in a table I can check."

Every number with the section and the sentence it came from

A flag on anything the checks could not confirm

Run on an indenture the course has never opened. Nothing promises a clean table: report what the checks show.

What the answer is not

Not a credit view on the issuer

Not a view on the notes

Not a ranking against another deal

Not a judgement of the terms

It describes what one public document says, and the tool refuses a request for more.

What you hand in: thirteen deliverables

#	Deliverable
1	textkit complete, pytest passing
2	load_indenture(path), headings checked
3	The indenture described, hashed checks
4	extract_covenant: one request per kind
5	check_table: a status for every value
6	run_tool, on Lamar and on Boyd
7	The Lamar table, failures beside quotes

#	Deliverable
8	The guardrail, asserted
9	Requests, tokens, and cost
10	A workbook with Excel formulas
11	What the tool shows, and where it fails
12	One commit per file, git tag course7
13	An AI-use log

The functions go into one module of your own, covenant_tool.py. textkit is frozen: the capstone adds nothing to it.

Two indentures, one route

The new one: never opened before today

Senior subordinated notes of 2007. A tool tested on the document it was tuned on has not been tested.

Boyd: the reference

Day 12's checked table sits beside the tool's run on Boyd: agreement on numbers, not proof of completeness.

The route: OpenAI, gpt-4.1-mini

The free model's daily requests were used up on the recording day (HTTP 429), so every request went through day 11's OpenAI route. Temperature 0, seed 42, room for 4,000 completion tokens, JSON asked for, the system prompt first.

Source: Lamar Media Corp., indenture for the 6-5/8% Senior Subordinated Notes due 2015, Series C, Exhibit 4.1 to the 8-K filed 2007-10-16, accession 0000950134-07-021393.

Source: Boyd Gaming Corporation, indenture for the 4.750% Senior Notes due 2031, Exhibit 4.1 to the 8-K filed 2021-06-08, accession 0001193125-21-185495.

What makes it hard: nothing is planted

Before the contents

something full of section numbers that is not a section

Housekeeping first

sections at the start of Article 4 that a desk does not record

Another debt test

written in words and forms day 12 never met

No kind in the schema

a covenant that is none of the seven kinds

Another measure

baskets sized against a measure day 12 never saw

A kind not there

a covenant kind the text may not hold at all

Each one has a question, and a closed block after it holds the decision the capstone expects.

Twelve sections, three sittings

First sitting

1 Your Complete Module

2 Loading an Indenture

3 The Indenture, Described

4 Article 4, Mapped

Load and describe

Second sitting

5 One Covenant, Extracted

6 The Checked Table

7 The Whole Tool

8 The Lamar Table

Extract and check

Third sitting

9 The Guardrail

10 Cost and Requests

11 The Workbook

12 What the Tool Shows

Guard, count, describe

In replay mode nothing goes out over the network: well under a minute, most of it reading two HTML files and the tests.

Load, then check the headings

find_sections keeps the last heading

The contents come first and the body's heading wins, but only if the pattern reads headings and nothing else. A wide pattern keeps cross-references inside paragraphs, with no error.

Two checks on every load

Every kept heading starts with Section

Article 4 runs 4.01, 4.02, ... in order

A failure stops before any request

```
text, sections, definitions = covenant_tool.load_indenture(LAMAR_FILE)
b_text, b_sections, b_definitions = covenant_tool.load_indenture(BOYD_FILE)
```

One function, one pattern, two indentures written years apart.

Describe before you send

3.1 Size and articles

words, characters, articles

3.2 Article 4

its sections; the housekeeping before the first covenant

3.3 The dictionary

definitions; the debt covenant's terms at depth 1 and 2

3.4 The debt test

its name, direction, and level, as written

3.5 The baskets

basket_amounts: the growers and their measure

3.6 A kind not there

titles and words of a suspension

Would day 12's number_forms find the debt test's level in its sentence? Work it out before any request.

Article 4, mapped

A kind of the schema

The section the tool extracts that kind from.

other

A later covenant with no kind in the schema, reported with its own first sentence.

not extracted

The housekeeping sections before the first covenant, listed.

The rule, written before the new document was opened: 7 kinds, a few title words each

A kind goes to the first section whose title holds one of its words; a later section with the same words is other. Nothing is dropped, nothing is forced.

One request per kind

One user message, three parts

1 The instruction: 5 sentences, chosen on Boyd

2 The definitions it uses, at depth 2

3 The section's text

The system prompt goes first; ask puts it there. The prompt never changes today.

extract_covenant, in order

No section: send nothing, return None

assert_extraction_only on the instruction

One request through ask, JSON asked for

NOT IN TEXT: say so

parse_json into a Covenant, or raise

Read finish_reason first, then the tokens, then the fields; end with the LLM line.

The checked table

The checks, row by row

section_exists, quote_found, number_found

the forms this document writes

direction_found, for a ratio

basket_agrees: basket_amounts, same section

A number written where the response says, not proof it is the right number for the field.

Three statuses, and no fourth

```
confirmed
```

```
not confirmed: kept, with its quote
```

```
not in text: an explicit absence
```

Allowed names

extracted value, with its quote

confirmed by the checks

Never called

a view on the notes

a judgement of the terms

The whole tool, and what is missing

run_tool

Load, map, one request per kind, the checks, and a hash of each response.

Boyd, beside day 12

Same covenant, unit, and number: agreement between two extractions, nothing more.

What did not come back

basket_amounts read every grower before any request: a key. Count what the table holds.

A table where every row is confirmed can still leave out the basket a reader needs most.

The guardrail, the cost, the workbook

The guardrail

Two adversarial cells: the request alone, and with a section after it. Decide what a test should accept.

The cost

Requests and tokens from the saved responses; usage_cost on the free model and at the fallback's list price.

The workbook

course7_capstone_covenants.xlsx: the checks as Excel formulas, read back, sections as text.

Allowed names

the response

the saved response

the model returned

Never called

the model's opinion or view

what the model believes

what the model knows

The description, Git, and the AI check

The description, 300 to 500 words

What the table says the indenture states

Which values the checks confirmed

What the checks cannot confirm

What the tool does not read

That nothing here is a view

Checked by assert_extraction_only. Then the AI-use log, a commit per file, and git tag course7.

The AI check: check_table

1. Docstring: confirmed only by its own quote
2. Code back: runs, a status for every row
3. Test first: a planted basket
4. Fix it, run the test again

Guess the assistant's status for the planted basket before you run the test.

What you can do now

- Clean, split, and count real financial text
- Embed and search an indenture by meaning
- Send a request with the key kept out of every cell
- Extract covenants into a schema and check every number
- Build a tool, 34 questions and 31 checks, on a new document

Next keep your my_bondmath folder up to date, and bring it with you.